

# Functions for working with a DOM element in JavaScript

Now we will learn how to make helper functions that perform any operations with the DOM. For example, let's make a function that takes the element's `id` as the first parameter, and the element's text as the second, and will set the new text for that element.

Here is the described function:

```
1 function setText(id, text) {  
2   let elem = document.getElementById(id);  
3   elem.textContent = text;  
4 }
```

Let's try out its work. Let's say we have two paragraphs:

```
1 <p id="elem1"></p>  
2 <p id="elem2"></p>
```

Let's change the text of these paragraphs using the function we created:

```
1 setText('elem1', 'text1');  
2 setText('elem2', 'text2');
```



## №1

⊗jsPmFDDEF

Modify the function I created so that it takes as a parameter not the `id` of the element, but an arbitrary CSS selector.

## №2

⊗jsPmFDDEF

Make the `setAttr` function that will change the element's DOM attribute. Let the function take the element selector as the first parameter, the attribute name as the second, and its new value as the third.

# Functions for working with a group of DOM elements

Let's now write a function that takes a group of elements selector and their new text as parameters. Let this function set new text to all elements, matching for the selector.

Let's implement the described function:

```
1 function setText(selector, text) {  
2   let elems = document.querySelectorAll(selector);  
3  
4   for (let elem of elems) {  
5     elem.textContent = text;  
6   }  
7 }
```

Let's test it for a group of the following elements:

```
1 <p class="elem"></p>
2 <p class="elem"></p>
3 <p class="elem"></p>
```

Set new text for all elements with class `elem`:

```
1 setText('.elem', 'text');
```

## № 1

⊗jsPmFDEGF

Make the `appendText` function that takes a selector as the first parameter and text as the second. Make this function append text to the end of passed elements.

# Passing a callback to work with the DOM in JavaScript

Suppose we have some group of DOM elements:

```
1 <p class="elem">1</p>
2 <p class="elem">2</p>
3 <p class="elem">3</p>
4 <p class="elem">4</p>
5 <p class="elem">5</p>
```

Let's make the function `forEach`, which will take the elements group selector as the first parameter, and the second parameter - a callback function, which will be applied to each of the found elements in turn:

```
1 forEach('.elem', function() {
2   // the function will be applied to each element
3 });
```

Let the elements that fall under the selector, in turn, fall into the first parameter of the callback:

```
1 forEach('.elem', function(elem) {
2   console.log(elem); // outputs the found elements one by one
3 });
```

Let's use our function to find all elements with the class `elem` and apply a callback for each element found, which will square the text of each element:

```
1 forEach('.elem', function(elem) {
2   elem.textContent = elem.textContent ** 2;
3 });
```

Let's now write the implementation of the `forEach` function:

```
1 function forEach(selector, func) {
2   let elems = document.querySelectorAll(selector);
```

```

3
4   for (let elem of elems) {
5       func(elem);
6   }
7 }

```

## № 1

⊗jsPmFDcP

Given paragraphs. Add an exclamation mark to the end of each paragraph text using the `forEach` function we created.

# Passing an index number to a callback in JavaScript

Let's make it so that if the second parameter is written in our callback function, let it contain the index number of the element in the found set of elements:

```

1  forEach('.elem', function(elem, index) {
2      console.log(elem); // outputs the element
3      console.log(index); // displays the element's index
4  });

```

With this parameter, we can, for example, add to the end of each element its index:

```

1  forEach('.elem', function(elem, index) {
2      elem.textContent = elem.textContent + index;
3  });

```

Let's refactor our `forEach` function code to implement the described:

```

1  function forEach(selector, func) {
2      let elems = document.querySelectorAll(selector);
3
4      for (let i = 0; i < elems.length; i++) {
5          func(elems[i], i);
6      }
7  }

```

## № 1

⊗jsPmFDcSNP

Given paragraphs. Using the function `forEach` we created for each paragraph, add its index to the beginning of it.

# Passing a DOM element as a function parameter

In previous lessons, we passed element selectors to our functions, and our functions themselves obtained references to these elements within their code. There is another approach: you can pass earlier obtained references to elements into functions.

Let's look at an example. Let's say we have the following code:

```

1 <p id="elem1"></p>
2 <p id="elem2"></p>

```

Let's make a function `setText`, which will take a reference to the DOM element as the first parameter, and the element's text as the second:

```

1 function setText(elem, text){
2   elem.textContent = text;
3 }

```

Using the function we created to set the text of our paragraphs:

```

1 let elem1 = document.getElementById('elem1');
2 setText(elem1, 'text1');
3
4 let elem2 = document.getElementById('elem2');
5 setText(elem2, 'text2');

```

## №1

⊗jsPmFDPEAP

Make the function `appendText` that takes a DOM element as the first parameter and text as the second. Make this function add text to the end of this element.

## №2

⊗jsPmFDPEAP

Given paragraphs. Get them, loop through and add '!' to the end of each using the `appendText` function made in the previous task.

## №3

⊗jsPmFDPEAP

Make the function `setValue`, which will take a reference to the input as the first parameter, and the text as the second. Make this function set the passed text to `value` of the input.

# Passing a group of DOM elements as function parameters

Now let our `setText` function take as the first parameter a reference not to single element, but to a set of elements at once:

```

1 function setText(elems, text) {
2   for (let elem of elems) {
3     elem.textContent = text;
4   }
5 }

```

Let's try out our function in practice. Suppose we have the following paragraphs:

```

1 <p class="elem"></p>
2 <p class="elem"></p>
3 <p class="elem"></p>
4 <p class="elem"></p>
5 <p class="elem"></p>

```

Let's use our function to set all these paragraphs to some text:

```
1 | let elems = document.querySelectorAll('.elem');
2 | setText(elems, 'text');
```

## №1

⊗jsPmFDPGAP

Make the `appendText` function that takes an array of DOM elements as the first parameter and text as the second. Make this function append text to the end of passed elements.

## №2

⊗jsPmFDPGAP

Make the `appendElem` function, which will take as the first parameter a reference to the DOM object that contains the `ul` tag, and as the second - the text. Make this function create a new `li` with the passed text and add it to the end of the passed `ul` tag.

## №3

⊗jsPmFDPGAP

Given an array and `ul`. Using the `appendElem` function created for the previous task, write each array element into a separate `li` in this `ul`.

# Function for creating HTML tables in JavaScript

Let's now make a function `createTable` that will create a table of the given size and add it to the end of the given element.

Let our function take the number of rows as the first parameter, the number of columns as the second parameter, and a reference to the DOM element in which our table will be created as the third parameter.

Let's see how we will use the described function when it is created. Let, for example, we have such a div:

```
1 | <div id="elem"></div>
```

Let's make a 3 by 4 table inside this div:

```
1 | let div = document.querySelector('#elem');
2 | createTable(3, 4, div);
```

Let now we have two divs:

```
1 | <div id="elem1"></div>
2 | <div id="elem2"></div>
```

Let's make own table in each of these divs:

```
1 let div1 = document.querySelector('#elem1');
2 createTable(3, 4, div1);
3
4 let div2 = document.querySelector('#elem2');
5 createTable(5, 6, div2);
```

In order for the created tables to be immediately visible, you can add some CSS, for example, this:

```
1 td {
2   width: 50px;
3   height: 50px;
4   border: 1px solid black;
5 }
```

## №1

⊗jsPmFDTCF

Here is a blank for the described function:

```
1 function createTable(rows, cols, parent) {
2   let table = document.createElement('table');
3
4   // here we'll create a table with rows (rows) and columns (cols)
5   for () {
6     for () {
7
8     }
9   }
10
11   parent.appendChild(table);
12 }
```

Add the code for the above function blank. Test the finished function.

# Returning a table from a function in JavaScript

In the previous lesson, we made the `createTable` function. Let's modify this function so that it does not add a table to some element, but simply returns it via `return`.

That is, the above code of the previous lesson is converted into this:

```
1 let div = document.querySelector('#elem');
2 let table = createTable(3, 4);
3 div.appendChild(table);
```

It can be rewritten shorter:

```
1 let div = document.querySelector('#elem');
2 div.appendChild(createTable(3, 4));
```

Getting a reference to the table may be needed in order to do something with the created table. For example, let's color its text color red:

```

1 let div = document.querySelector('#elem');
2
3 let table = createTable(3, 4);
4 table.style.color = 'red';
5
6 div.appendChild(table);

```

## № 1

⊗ jsPmFDRT

Modify your `createTable` function as described in theory.

## № 2

⊗ jsPmFDRT

Suppose we have such a div with paragraphs:

```

1 <div id="elem">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5 </div>

```

Use the `createTable` function to create a new table and then insert it at the end of the div.

# Function to create a table from an array

In the previous lessons, we made the `createTable` function, which creates a table of a given size. Let's now make the `createTableByArr` function, which will take a two-dimensional array as a parameter and build a table based on it.

That is, the above code of the previous lesson is converted into this:

```

1 let div = document.querySelector('#elem');
2
3 let arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
4 let table = createTableByArr(arr);
5
6 div.appendChild(table);

```

The result of executing this code should be the following table:

```

1 <div id="elem">
2   <table>
3     <tr>
4       <td>1</td>
5       <td>2</td>
6       <td>3</td>
7     </tr>
8     <tr>
9       <td>4</td>
10      <td>5</td>
11      <td>6</td>
12    </tr>

```

```
13     <tr>
14         <td>7</td>
15         <td>8</td>
16         <td>9</td>
17     </tr>
18 </table>
19 </div>
```

## № 1

⊗jsPmFDTcFA

Implement the described function. Check out its work.